

Výstupy ŠVP

INF-07a Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešení problém ten nejvhodnější. (Informatika 4, 2/2 · z INF-07)

INF-07b Vylepším algoritmus podle zvoleného hlediska a zobecním řešení pro širší třídu problémů. (Informatika 4, 2/2 · z INF-07)

INF-08d Ve snaze o vyšší efektivitu navrhuji, řídím a hodnotím souběh procesů. (Informatika 4, 2/2 · z INF-08)

Učivo: testování, optimalizace – vliv vstupních dat na spotřebované výpočetní zdroje

Základní pojmy

Časová náročnost – Jak roste počet kroků algoritmu s velikostí vstupu n : logaritmičtý ($\log_2 n$), lineárně (n), kvadraticky (n^2).

Lineární vyhledávání – Prochází prvky po řadě; nejhůř n porovnání; funguje i pro neseřazená data.

Binární vyhledávání – V seřazeném seznamu porovná prostřední prvek a zahodí polovinu; každé porovnání = 1 bit informace (H34).

Řazení výběrem – Opakovaně najde nejmenší prvek zbytku a přesune ho na začátek; vždy $\frac{n(n-1)}{2}$ porovnání.

Bublínkové řazení – Porovnává sousedy a prohazuje; vylepšení: skončit, když v průchodu nedošlo k výměně.

Vylepšení algoritmu – Úprava podle zvoleného hlediska (rychlost, paměť, čitelnost) doložená měřením nebo počtem kroků – se stejnými testy.

Zobecnění – Pevné hodnoty nahradím parametry (hledaný prvek, směr řazení) a řešení zabalím do funkce použitelné pro jiná data.

Souběh procesů – Více procesů (vláken, jader, postav ve Scratchi) pracuje současně; zrychlí práci, kterou lze rozdělit na nezávislé části.

Kolize nad sdílenými daty – Výsledek závisí na tom, jak se kroky procesů prostřídají (načti – změň – zapiš); řeší se zámek, čekáním na zprávu, střídáním.

Uvážnutí (deadlock) – Procesy čekají jeden na druhý a žádný nemůže pokračovat.

Vzorce a vztahy

$2^k \geq n + 1$ binárnímu vyhledávání stačí k porovnání (nejmenší takové k); $n = 10^6 \Rightarrow k = 20$

$\frac{n(n-1)}{2}$ počet porovnání řazení výběrem (a bublínkového bez vylepšení)

$n \log_2 n$ řádový počet porovnání vestavěného sorted()

$S = \frac{t_1}{t_p}$ zrychlení souběhem: čas na 1 procesoru / čas na p procesorech; vždy $S \leq p$

$t_p = t_{\text{sekv}} + \frac{t_{\text{děl}}}{p}$
část, kterou nelze rozdělit, zrychlení omezuje

Počet porovnání v nejhorším případě

n	lineární vyhledávání	binární vyhledávání	řazení výběrem	$\approx n \log_2 n$
10	10	4	45	33
1 000	1 000	10	499 500	$\approx 10 000$
10^6	10^6	20	$\approx 5 \cdot 10^{11}$	$\approx 2 \cdot 10^7$

Poučky a postupy

- Algoritmy porovnávej podle počtu kroků v nejhorším případě, naměřeného času, paměti a čitelnosti – vždy pro několik rostoucích n .
- Zdvojnásobení n : lineární algoritmus $2\times$ déle, kvadratický $4\times$, binární vyhledávání jen o 1 krok víc.
- Binární vyhledávání potřebuje seřazená data; řazení stojí víc než jedno lineární hledání – vyplatí se, když hledám mnohokrát.
- Měř čas pro více velikostí vstupu a sleduj poměry; jedno měření nic neřká.
- Vylepšení dolož: stejné testy před i po úpravě + počet kroků nebo čas.
- Souběh: rozděľ práci na nezávislé části, sdílená data chraň (zámek, čekání na zprávu); zrychlení omezuje nedělitelná část a režie.

Pozor na

- Binární vyhledávání v neseřazeném seznamu vrací špatné výsledky – bez chybové hlášky.
- Kvadratický algoritmus je pro 100 prvků „rychlý“; problém se ukáže až pro miliony.
- 4 jádra $\neq 4\times$ rychleji: část práce je sekvenční a procesy se musí domlouvat.
- Měření času ruší jiné programy – měř víckrát, porovnávej poměry.

Vzorový příklad

Kolik porovnání potřebuje binární vyhledávání v seřazeném seznamu 1 000 000 jmen a kdy se ho vyplatí použít?

- Každé porovnání s prostředním prvkem zmenší interval na polovinu (1 bit informace).
- Stačí nejmenší k s $2^k \geq n + 1$: $2^{19} = 524 288 < 10^6$, $2^{20} = 1 048 576 \rightarrow k = 20$.
- Lineární vyhledávání potřebuje nejhůř 10^6 porovnání $\rightarrow$ binární je v nejhorším případě $50 000\times$ rychlejší.
- Seznam ale musí být seřazený: řazení stojí asi $n \log_2 n \doteq 2 \cdot 10^7$ porovnání.
- Vyplatí se tedy, když v týchž datech hledám mnohokrát (asi od 40 hledání – úloha 13).